2026-06-07 - Work Queues and Kernel Timers

Goal

Learn how to schedule deferred and periodic work in Zephyr using kernel timers and work queues, without blocking threads or polling in a tight loop.

What you will learn

- How `k_timer` fires a callback at a set interval
- How `k_work` defers processing out of ISR / timer context
- How `k_work_delayable` schedules work after a delay
- How to create a dedicated work queue thread for heavy background tasks

Overview

Timers and work queues are Zephyr's core tools for time-driven logic. A timer fires in interrupt context — keep the callback short. Hand off real work to a `k_work` item that runs in a thread.

One-shot and periodic timers

```
static void timer_cb(struct k_timer *timer) {
    /* runs in ISR context — keep it short */
    printk("tick\n");
}

K_TIMER_DEFINE(my_timer, timer_cb, NULL);

/* Start: fire once after 2 s */
k_timer_start(&my_timer, K_SECONDS(2), K_NO_WAIT);

/* Start: fire every 500 ms after an initial 1 s delay */
k_timer_start(&my_timer, K_SECONDS(1), K_MSEC(500));

/* Stop */
k_timer_stop(&my_timer);
```

Submitting work from a timer

```
static struct k_work my_work;
```

```
static void work_handler(struct k_work *work) {
    /* runs in system work queue thread — safe to call kernel APIs */
    LOG_INF("Work executed at uptime %lld ms", k_uptime_get());
}

static void timer_cb(struct k_timer *timer) {
    k_work_submit(&my_work);  /* safe to call from ISR */
}

K_TIMER_DEFINE(my_timer, timer_cb, NULL);

void main(void) {
    k_work_init(&my_work, work_handler);
    k_timer_start(&my_timer, K_NO_WAIT, K_SECONDS(1));
}
```

Delayable work (self-rescheduling)

```
static struct k_work_delayable dwork;

static void dwork_handler(struct k_work *work) {
    LOG_INF("Deferred work");
    /* reschedule itself */
    k_work_reschedule(&dwork, K_MSEC(500));
}

void main(void) {
    k_work_init_delayable(&dwork, dwork_handler);
    k_work_schedule(&dwork, K_NO_WAIT);
}
```

Custom work queue (dedicated thread)

```
K_THREAD_STACK_DEFINE(my_wq_stack, 1024);
static struct k_work_q my_wq;

k_work_queue_start(&my_wq, my_wq_stack,
                   K_THREAD_STACK_SIZEOF(my_wq_stack),
                   K_PRIO_PREEMPT(5), NULL);

/* Submit to your queue instead of the system queue */
k_work_submit_to_queue(&my_wq, &my_work);
```

Use a dedicated queue when work is heavy, slow, or must not block the system work queue that other Zephyr subsystems rely on.

Checking timer remaining time

```
k_ticks_t remaining = k_timer_remaining_ticks(&my_timer);
int64_t   ms        = k_ticks_to_ms_floor64(remaining);
```

Why this matters

Work queues are used throughout Zephyr internals — the BLE stack, USB stack, and sensor trigger system all post work items. Understanding them lets you build responsive, non-blocking firmware without complex thread management.

Practice tasks

1. Create a 1 Hz timer that submits work to toggle `led0`.
2. Add a `k_work_delayable` that prints uptime every 3 s without a timer.
3. Create a dedicated work queue and move heavy logging there.

Example folder

See `src/2026-06-07_Work_Queues_and_Timers` for the day's code.

Next topic

Tomorrow we configure Zephyr's power management to put the nRF52840 into low-power sleep between work items.